

Week 2: analytical platform design

1. Analytical requirements

The Week 2 platform supports repeated analysis of New York taxi demand, weather and air quality. It builds on the Week 1 Spark and Delta Lake pipeline, using its cleaned Silver data rather than repeating ingestion. The main input is the integrated table: one row per accepted trip, with pickup location, calendar fields and environmental context. Clean taxi trips and the zone lookup are also loaded for the join experiment.

The analytical requirements are six reproducible questions with explicit units of comparison:

Analysis	Required answer
Monthly zone demand	Pickup counts for each month and pickup zone.
Weather and distance	Average trip distance in dry, wet and unknown conditions.
Air quality and demand	Association between hourly citywide PM2.5 and hourly pickups.
Weather demand variation	Zones with the largest difference in pickups per wet versus dry hour.
Weekday peaks	Busiest hours for each weekday, preserving tied peaks.
Monthly trends	Monthly totals, daily averages and change from the previous month.

Demand means the number of pickups, not passenger count or revenue. Calendar analyses use America/New_York; environmental observations are matched using UTC hours. Counts, coverage and missing values must remain visible so that an apparent change in demand is not automatically interpreted as an environmental effect.

The engineering requirements are to answer all six questions directly from Silver, create four reusable Gold data products, and compare equivalent results before and after optimization. A successful optimization must preserve the answer. The implementation must also expose its SQL and physical plans so another reader can reproduce and explain the measurements.

2. Analytical query design

Shared preparation

The processing path is **Silver tables -> shared SQL views -> analytical answers and Gold summaries**. Temporary views are logical definitions; a cache or Delta write is required to materialize their results.

The `trips` view projects the required columns and derives the UTC observation hour and precipitation category. A matched precipitation value of zero is dry, a positive value is wet, and missing or unmatched precipitation is unknown. This classification intentionally measures precipitation rather than temperature, snowfall or weather intensity.

An hourly calendar covers the first pickup date through the end of the last pickup date. It is generated in UTC between New York local midnights, then mapped back to local date, weekday and hour. Spring and autumn daylight-saving transitions therefore contain 23 and 25 actual hours respectively. The `hourly_demand` view joins trip counts onto this calendar and assigns zero to hours without trips.

This denominator assumes a complete release over the covered dates. An input outage cannot be distinguished from genuinely zero demand. Coverage statistics retain total trips, first and last dates, days with trips and unknown-weather trips. Environmental context comes from integrated trip rows, so an hour with no trips anywhere in the city has unknown context even if an external observation might exist.

Query definitions and interpretation

Monthly zone demand. Group by the month date, pickup location ID and location labels, then count trips. A month date preserves the year and avoids combining January from different years. Only zone/month combinations containing trips are returned; unused lookup locations are not treated as observed zero-demand zones.

Weather and trip distance. Group trips by dry, wet or unknown precipitation and calculate mean distance. Return both trip count and non-null distance count so the denominator is explicit. Missing weather remains a separate category. This query describes differences between groups; it does not attribute changes in distance to weather.

Air quality and demand. Aggregate to one observation per UTC hour before calculating Pearson correlation between demand and citywide PM2.5. Use `pickup_pm25_citywide`, rather than the convenience field that combines borough estimates with citywide fallback values. Giving each hour one observation prevents busy hours from receiving extra weight merely because their PM2.5 value appears on more trip rows. Report total and matched hours. Return null correlation for fewer than two matched observations or zero variance. Time of day, weekday and season remain potential confounders.

Weather-related demand variation. For each zone, divide trips in a weather category by the number of citywide hours observed in that category. Include category exposure even when that zone has no trips; weather from trips in other zones can establish the city's context. The `zone_weather` view combines observed zones, category exposure hours and trip totals to make these zero counts explicit.

For zones exposed to both dry and wet conditions, rank the absolute difference between their two demand rates. This measures the size of the difference, not its direction. The smaller exposure count is returned as `minimum_exposure_hours` to identify sparse comparisons. Absolute differences tend to favor busy zones and are not significance tests. Exposure hours repeat across zone rows and must not be summed across zones.

Weekday peak hours. Average hourly demand over the actual occurrences of each local weekday/hour combination, including zero-demand hours. Apply dense ranking within each weekday. Returning all ranks supports inspection of the full daily profile; rank 1 identifies peaks and retains ties. The repeated autumn hour contributes two actual observations.

Monthly demand trends. Sum pickups by month and count covered calendar days. Report average daily demand, the previous month's trip count and percentage change using a window function. Percentage change is null if the previous month is missing or zero. Observed-day counts identify partial months, whose totals should be interpreted with their coverage.

SQL organization and correctness

SQL files are separated into `views/`, `silver/`, `products/`, `gold/` and `benchmarks/` under `src/week2/sql/`. Matching Silver and Gold filenames make each pair easy to compare. Python handles loading, execution, table writes and timing. Small controlled placeholders supply calendar dates, the selected benchmark month and the broadcast hint.

Results are matched by business keys rather than row order. Integer counts and null values must agree exactly; floating-point aggregates allow relative tolerance $1e-9$ and absolute tolerance $1e-8$ for summation-order differences. Known-answer fixtures cover all six queries, missing observations, zero-demand hours, daylight-saving changes and ties. Product tests also check Silver/Gold equivalence and repeatable refreshes.

3. Analytical data products

Gold stores common aggregations so users can repeat analyses without rescanning millions of individual trips. Four products support the six analytical questions.

Product	One row represents	Measures and intended use
Daily Mobility Summary	Local day and borough	Trip count; distance, duration and fare sums and valid counts. Supports transport planners and monthly trend reports.
Taxi Zone Statistics	Month and pickup zone	Trip count, zone labels, distance and fare sums and valid counts. Supports zone demand and cost comparisons.
Weather Impact Summary	Zone and precipitation category	Trips, exposure hours, distance sum and valid-distance count. Supports weather-distance and weather-demand analyses.
Air Quality Impact Summary	UTC hour	Demand, citywide PM2.5, local calendar fields and weather category. Supports air-quality correlation and weekday peaks.

The physical names are `daily_mobility_summary`, `taxi_zone_statistics`, `weather_impact_summary` and `air_quality_impact_summary`. They are written beneath `storage/delta/gold/week2_simple/`.

The daily product supplies monthly trend totals; the shared calendar supplies covered-day counts. The zone product directly supplies monthly zone demand. The weather product supports two queries by combining its zone-level sums and counts or comparing category demand rates. The hourly product supports both air-quality correlation and weekday profiles.

Preserve meaning when reaggregating

Products retain additive sums and non-null counts rather than only storing averages. To calculate an overall average distance, sum the distance totals and divide by the summed valid-distance counts. Averaging group averages would overweight small groups.

Not every field is additive. In particular, weather exposure hours describe a common citywide calendar and repeat across zones. Consumers must use them as denominators for each zone, not sum them as if they represented different hours. The product definitions and Gold queries make that distinction explicit.

Physical layout and refresh

At the measured scale, each Gold product contains only hundreds or a few thousand rows. One unpartitioned Delta data file per product avoids many tiny partitions and keeps reads simple. This layout is a scale-dependent decision: larger histories or multiple cities could require different partition and file sizes.

Refresh uses a full overwrite of each current snapshot. Repeating a build against unchanged input produces the same row contents, while Delta history is retained. A four-row metadata table records each product's source, creation time, refresh time, schema version and row count. Initial creation time survives subsequent refreshes. The schema version is explicitly updated when a product definition changes.

Silver must remain unchanged during execution. Users rebuild Gold after modifying Silver data or SQL; the `benchmark` command always rebuilds Gold first. Each product and the metadata commit separately, so a failed build can leave different refresh stages. Recovery is to rerun the product build before consuming Gold.

Automatic freshness detection, source-version pinning, concurrent refresh writers and incremental updates are omitted to keep the course implementation understandable. The trade-off is a clear operational responsibility: a saved Gold table must not be assumed current merely because it exists.

4. Optimization strategy

The benchmark separates preaggregation from individual Spark techniques. All six analyses are measured over Silver and Gold with Adaptive Query Execution (AQE) and automatic broadcasting disabled. Four focused comparisons then select one Silver query per technique.

Technique	Controlled comparison
Caching	Run weather-distance SQL before and after caching the projected trips view. Measure cache population separately.
AQE	Run weather-demand variation with AQE and shuffle coalescing off, then on. Keep automatic broadcasting disabled.
Partition pruning	Count pickups per zone for one month using date bounds alone, then add explicit year/month partition filters.
Broadcast join	Join clean taxi trips to zone information without a hint, then broadcast the small zone lookup.

Both pruning queries use the same already partitioned integrated table and return the same monthly answer. This tests the additional benefit of explicit partition predicates, not partitioned versus unpartitioned storage. Date-based Delta file skipping can already eliminate irrelevant files, so a faster explicit-filter result is not assumed.

The broadcast experiment uses the underlying clean tables because integrated trips already contain zone attributes. Its baseline must avoid a broadcast hash join, and the hinted plan must contain one. Both variants are also checked against the integrated monthly-zone answer. Caching must produce a cached scan. AQE plans are inspected after execution to reveal adaptive shuffle decisions.

5. Engineering decisions and trade-offs

Measure complete answers. Each case has one warmup and five timed executions. Timing includes planning, execution and collection of the complete answer, ensuring analytical calculations are performed. Correctness checks and plan extraction are outside the timer; Gold construction and cache population are recorded separately.

Keep performance evidence inspectable. The 18 cases comprise 12 Silver/Gold cases and six additional focused cases, with two baselines reused. The default gives 90 timed executions. JSON stores samples, statistics, SQL, plans and status; Markdown reports are maintained manually. Failed runs retain partial measurements and must not support a complete comparison.

Separate query latency from ownership cost. Gold summaries require storage and refresh work. Caching needs repeated savings to repay population cost and uses executor memory. Broadcasting avoids join-side shuffles but requires a lookup that fits in memory; later aggregation can still shuffle. AQE helps only where runtime information enables a useful change. None of these techniques guarantees a benefit for every workload.

Limit conclusions to the experiment. Runs use a fixed order and warm operating-system and JVM caches. Spark caches are cleared between relevant phases. Medians and timing spread describe this local workload, not statistical significance or distributed-cluster performance. Physical plans identify operators, but do not establish actual file-read counts.

Prepare for growth without adding unused machinery. For ten cities, add `city_id` to keys and groupings, use local calendars and retain UTC joins. Reassess partition sizes and memory budgets. Incremental refresh could update affected city/date slices rather than rebuilding every product.

The [benchmark report](#) records the measured outcomes and plan evidence. The [README](#) provides the commands needed to reproduce them.